Traineeship Application

Sprint Report

<theInvincibles>
<Dimitra Christina Gkaravela AM: 5051,
Nikoleta Tourounoglou AM:5106>

VERSIONS HISTORY

Date	Version	Description	Author
23/05/2025	0.0.1	Implementation of the Traineeship Management Application	Nikoleta Tourounoglou Dimitra Christina Gkaravela

1 Introduction

This document provides information concerning the **5** sprint of the project.

1.1 Purpose

We are building a web app for managing traineeships as our software engineering project. It lets students set up profiles and apply for internships, companies post and manage openings, and professors oversee placements, and a committee decides and reviews the traineeship with a PASS/FAIL mark.

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes out Scrum team and specifies the this Sprint's backlog. Section 3 specifies the main design concepts for this release of the project.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	Nikoleta Tourounoglou
Scrum Master	Dimitra Christina Gkaravela
Development Team	Nikoleta Tourounoglou, Dimitra Christina Gkaravela

2.2 Sprints

Sprint No	Begin Date	End Date	Number of weeks	User stories
01	**4/3/2025**	**5/3/2025**	**0**	**US1, US2**
02	**5/3/2025**	**6/3/2025**	**0**	**US4, US7, US13, US3**
03	**1/5/2025**	**8/5/2025**	**1**	**US5, US6, US8, US10, US11**

04	**9/5/2025**	**16/5/2025**	**1**	**US16, US17, US18, US19**
05	**16/5/2025**	**23/5/2025**	**0**	**US14, US9, US12, US15, US20, US21**

3 Use Cases

3.1 <Register>

Use case ID	01
Actors	The user
Pre conditions	The user has opened the website.
Main flow of events	1. The use case starts when the user wants to register. 2. If the user selects “Register” 3. The system displays the user’s information fields: ‘Username’, ‘Password’ and ‘Role’ 3.1 The user selects student in the role field, if they want to be a ‘Student’ 3.2 The user selects company in the role field, if they want to be a ‘Company’ 3.3 The user selects professor in the role field, if they want to be a ‘Professor’ 3.4 The user selects committee in the role field, if they want to be a ‘Committee’ 4. The system checks the information given and it displays a message that the user is successfully registered
Alternative flow 1	If the user is already registered, the system displays the message: “You are already registered”.
Alternative flow 2	If the user doesn’t fill any of the above fields the system displays the message: “Please fill out the entire form”.
Post conditions	The user is registered.

3.2 <Login >

Use case ID	02
Actors	The user
Pre conditions	The user already has an account.
Main flow of events	1. The use case starts when the user wants to log in. 2. The user selects “Login” 3. The system displays the login form with the fields “Username” and “Password”. 4. The user fills in form with their credentials and presses ‘Login’. 5. The system verifies the account. 6. The system redirects the user to the equivalent dashboard depending on the role of the user.
Alternative flow 1	If the user has falsely filled the fields the system displays an error message.
Post conditions	The user has successfully logged in and is the equivalent dashboard depending on the role.

3.3 <Logout >

Use case ID	03
Actors	The user
Pre conditions	The user has already logged in.
Main flow of events	1. The use case starts when the user wants to log out. 2. The system terminates the user’s interaction with the traineeship management application.
Post conditions	The user has logged out.

3.4 <Create Profile > US4, US7, US13,

Use case ID	04
Actors	The user, The student, The company, The professor
Pre conditions	The user has already logged in to the system.
Main flow of events	1. The use case starts when the user wants to create a profile. 2. The user selects the “My Profile” button on the Dashboard. 3. If the Role of the user is a student 3.1. The system displays the form with the fields: Full name, University ID Number, a list of interests separated by comma, a list of skills separated by comma, and a preferred location where the user would like to pursuit their traineeship. 4. If the Role of the user is a company 4.1. The system displays the form with the fields: Company Name and Location of the company. 5. If the Role of the user is a professor 5.1 The system displays the form with the fields: Full name and a list of interests separated with comma. 6. The user presses “Save” and the system redirects the user to the equivalent dashboard.
Alternative flow 1	At any time the user may go back to the Dashboard by pressing the button “Cancel”.
Post conditions	The user has created their profile successfully and is redirected to the dashboard.

3.5 <Apply for traineeship>

Use case ID	05
Actors	The user, The student
Pre conditions	The user has logged in and created their profile successfully.
Main flow of events	1. The use case starts when the user wants to apply for a traineeship. 2. The user selects the “Browse Position” section on the student

	dashboard. 3. The system transfers the user to a list of available traineeship positions that displays the information of the traineeship such as the title of the traineeship, the name of the company, the location of the company and the required skills needed to get selected for the traineeship. 3.1 If the list is empty, the system displays a "*No open positions available*" message. 4. The user selects the "Apply for Traineeship" button to apply for a traineeship. 5. The system transfers the user to the student dashboard.
Alternative flow 1	The user can return to student dashboard by selecting the "Back to Dashboard" button without applying for a traineeship.
Post conditions	The user's application has been submitted.

3.6 <updateLogbook>

Use case ID	06
Actors	The user, The student
Pre conditions	The user has logged in and created a profile.
Main flow of events	1. The use case starts when the user wants to report what they did in the traineeship. 2. The user selects the "My Logbook" button on the student dashboard. 3. The system presents to the user the form of the logbook 3.1 The form is empty and the system displays the "*No entries yet.*" Message. 3.2 The form has already some data filled out by the user sorted by the most recent entry. 4. The user selects the "New Entry" button. 5. The system displays a new logbook entry form with the message "What did you do today?". 6. The user fills in a report on what they did. 7.1 The user selects save. 7.2 The user selects cancel.

	8. The system redirects the user back to the logbook section. 9. The user selects the “Back to Dashboard” button. 10. The system redirects the user to the student dashboard.
Alternative flow 1	If the user tries to save an empty logbook entry, the system displays an error message “Please fill out this field”.
Post conditions	The user has filled in the logbook.

3.7 <ShowAvailableTraineeshipPositionsAdvertised>

Use case ID	07
Actors	The user, The Company
Pre conditions	The user has logged in and created a profile.
Main flow of events	1. The use cases starts when the user wants to access the list of available traineeship positions that they have advertised. 2. The user selects the “My Advertised Positions” from the company dashboard. 3. The system displays the advertised positions list. 3.1 If the list is empty, the system displays a “*No open positions.*” message. 3.2 If the list has already some positions available, the system displays the title of the position of the traineeship, the start and the end of the traineeship, the required skills for the traineeship and action section where they can delete the position. 4. The user adds a new position by pressing the “+New Position” button. 5. The user returns to dashboard by pressing the “Back to dashboard” button.
Post conditions	The user has accessed the list of the available positions.

3.8 <addNewPosition>

Use case ID	08
Actors	The user, The company

Pre conditions	The user has logged in, created a profile and selected “My Advertised Positions” section from the dashboard.
Main flow of events	1. The use cases starts when the user wants to add a new traineeship position. 2. The user selects the “+New Position” button. 3. The system displays the “Announce a New Position” form with the fields Title, Start Date, End Date, Description, Required Skills and Topics of Interest. 4. The user fills in all fields. 5. The user has the option: 5.1 The user saves the announce position by pressing the “Announce position” button. 5.2 The user cancels the announcement of the new position by pressing the “Cancel” button.
Alternative flow 1	If the user doesn’t fill in a field, the system displays a “Please fill out this field.” message on the according empty field.
Post conditions	A new available position has been announced.

3.9 <deleteTraineeshipPosition>

Use case ID	09
Actors	The user, The company
Pre conditions	The user has logged in, created a profile, selected “My Advertised Positions” section from the dashboard and there is already a traineeship position announced.
Main flow of events	1. The use cases starts when the user wants to delete a traineeship position. 2. The user selects the “Delete” button from the “Actions” section. 3. The system displays a notification asking the user whether they are sure to delete this position. 3.1 The user selects “Ok” where the position is deleted and the system

	displays a “Position deleted successfully” message. 3.2 The user selects “Cancel” and the system cancels the deletion.
Alternative flow 1	The user can go back to dashboard at any time by selecting the “Back to dashboard” button.
Post conditions	The traineeship position has been deleted.

3.10 <evaluateAssignedPositions>

Use case ID	10
Actors	The user, The company
Pre conditions	The user has logged in, has created a profile and the committee has assigned a traineeship to the student.
Main flow of events	1. The use cases starts when the user wants to evaluate a traineeship position that has been assigned to a student. 2. The user selects the “Assigned Positions” section from the company dashboard. 3. The system displays the list of traineeship positions that have been assigned to students featuring the title of the traineeship, the student username, the start and end date, a small description of the traineeship and the field “Actions” with the choice evaluate. 3.1 If the list is empty, the system displays “No positions have yet been assigned.” message. 3.2 If the list already has some positions that have been assigned to students, the user has the option to evaluate. 4. The user selects “Evaluate” button from the “Actions” section. 5. The system displays the evaluation form showing the evaluation fields rating the Motivation, the Effectiveness and the Efficiency of the student from a scale from 1 (bad) to 5 (perfect). 6. The user selects “Save Evaluation” saving the evaluation. 6.1 The user selects “Back to Assigned” button. 7. The system redirects the user back to “Assigned Positions” section.

Alternative flow 1	The user can return to dashboard at any time by selecting the “Back to Dashboard” button.
Alternative flow 2	The user can return to available positions when in “Assigned Positions” by selecting the “Back to Available” button.
Post conditions	The user has evaluated a traineeship position that has been assigned to a student.

3.11 <evaluateSupervisedTraineeships>

Use case ID	11
Actors	The user, The professor
Pre conditions	The user has logged in and has created a profile and the committee has assigned him as a supervisor in a traineeship position.
Main flow of events	1. The use cases starts when the user wants to evaluate a traineeship position that they supervise. 2. The user selects “My Supervised Positions” section from the professor dashboard. 3. The system displays the supervised traineeships of the professor with the fields: title of the traineeship, company name, student username, period of traineeship, description of the traineeship and “Actions” with the evaluation button. 4. The user selects “Evaluate” button from section “Actions”. 5. The system displays the evaluation form, rating the student’s motivation, effectiveness and efficiency and the hosting company’s facilities and guidance on a scale from 1 (bad) to 5 (perfect).. The system also displays the name of the traineeship, the username of the student and the name of the company. 6. The user fills in all fields. 6.1 The user selects Submit. 6.1.1The system displays “Evaluation saved!” message. 6.2 The user selects Cancel. 7. The system redirects the user to the “My Supervised Traineeships” section.

Alternative flow 1	The user can go back to professor dashboard with the Back button.
Post conditions	The professor has evaluated the student motivation, effectiveness and efficiency and the hosting company in terms of the facilities provided and the guidance of the trainee student on a scale from 1 (bad) to 5 (perfect).

3.12 <searchTraineeshipPosition>

Use case ID	12
Actors	The user, The committee
Pre conditions	The user has logged in.
Main flow of events	1. The use cases starts when the user wants to search for a list of available traineeship positions for a student 2. The user selects “List Student Applications” section from the committee dashboard. 3. The system displays the students that have requested a traineeship showing the student’s name, their university ID and the section “Actions” with the 3 different search strategies. 4. The user chooses a strategy 4.1 The user chooses search by interests 4.2 The user chooses search by location 4.3 The user chooses search by their intersection (both) 5. The system displays the results of the search of the traineeship positions for the student with the fields: title of the traineeship, name of the company, location of the company, the required skills to get accepted for the position and the “Action” section so that the user can assign a position to a student. 6. The user selects the “Back to Requests” button to return to “List Student Applications”.
Alternative flow 1	If the system doesn’t find any matches it displays “No positions available” message.
Post	The system has found a list of available traineeship positions for a student.

conditions	

3.13 <assignTraineeshipPosition>

Use case ID	13
Actors	The user, The committee
Pre conditions	The user has logged in, selected “List Student Applications” section and chose a search strategy for a student.
Main flow of events	1. The use cases starts when the user wants to assign a traineeship position to a student. 2. The user selects the “Assign” button. 3. The system assigns a position to the student and redirect the user to the “List Student Applications”.
Post conditions	The committee has assigned a traineeship position to a student.

3.14 <assignSupervisor>

Use case ID	14
Actors	The user, The committee
Pre conditions	The user has logged in and a traineeship position has been assigned to a student.
Main flow of events	1. The use cases starts when the user wants to assign a supervisor professor to a traineeship position. 2. The user selects the “Assign Supervising Professor” section from the committee dashboard. 3. The system displays the list of positions waiting for supervisor with fields the title of the traineeship, the company’s name, the dates of the traineeship and the “Actions” section with the “Assign Supervisor” button. 4. The user selects the “Assign Supervisor” button. 5. The system displays the list of professors that can be a supervisor to a

	traineeship with two search strategies, one based on the professor's interests and one based on the professor's load. 5.1 The user chooses a supervisor based on the professor's interests. 5.2 The user chooses a supervisor based on the professor's load. 6. The system displays the list of the professors sorted based on the user's selected search strategy. 7. The user selects a professor as a supervisor. 8.1 The user selects the "Assign Supervisor" button. 8.1.1 The system displays a "Supervisor assigned successfully!" message. 8.2 The user selects the "Cancel" button. 9 The system redirects the user to the "Assign Supervising Professor" section.
Alternative flow 1	If the user want to go back to dashboard they can select the back to dashboard button.
Post conditions	The committee has assigned a supervisor professor to a traineeship.

3.15 <monitorEvaluations>

Use case ID	15
Actors	The user, The committee
Pre conditions	The user has logged in, the professor and the company have evaluated a specific traineeship.
Main flow of events	1. The use cases starts when the user wants to monitor a traineeship position. 2. The user selects "List In-Progress Traineeships" section from the committee dashboard. 3. The system displays a list of the in-progress traineeships with the fields: the title of the traineeship, company name, student username, dates of the traineeship, the average score of the company's evaluation, the average score of the professor's evaluation, the status of the traineeship and "Actions" with the Review button. 4. The user selects the "Review" button.

	5. The system displays the company and professor's evaluations fields and the total average grade with a suggested final decision. 6. The user selects if the status of the traineeship is pass or fail. 7.1The user selects "Save Decision" button completing the traineeship. 7.1.1 The system displays a "Traineeship marked PASS/FAIL" 7.2 The user selects "Cancel". 7. The system redirects the user to the in-progress traineeships' list.
Alternative flow 1	The user can go back to committee dashboard with the "Back to dashboard" button.
Alternative flow 1	The system displays an error message when the users haven't marked the traineeship as pass or fail.
Post conditions	The committee completed the whole process with a pass or fail mark depending on the evaluations.

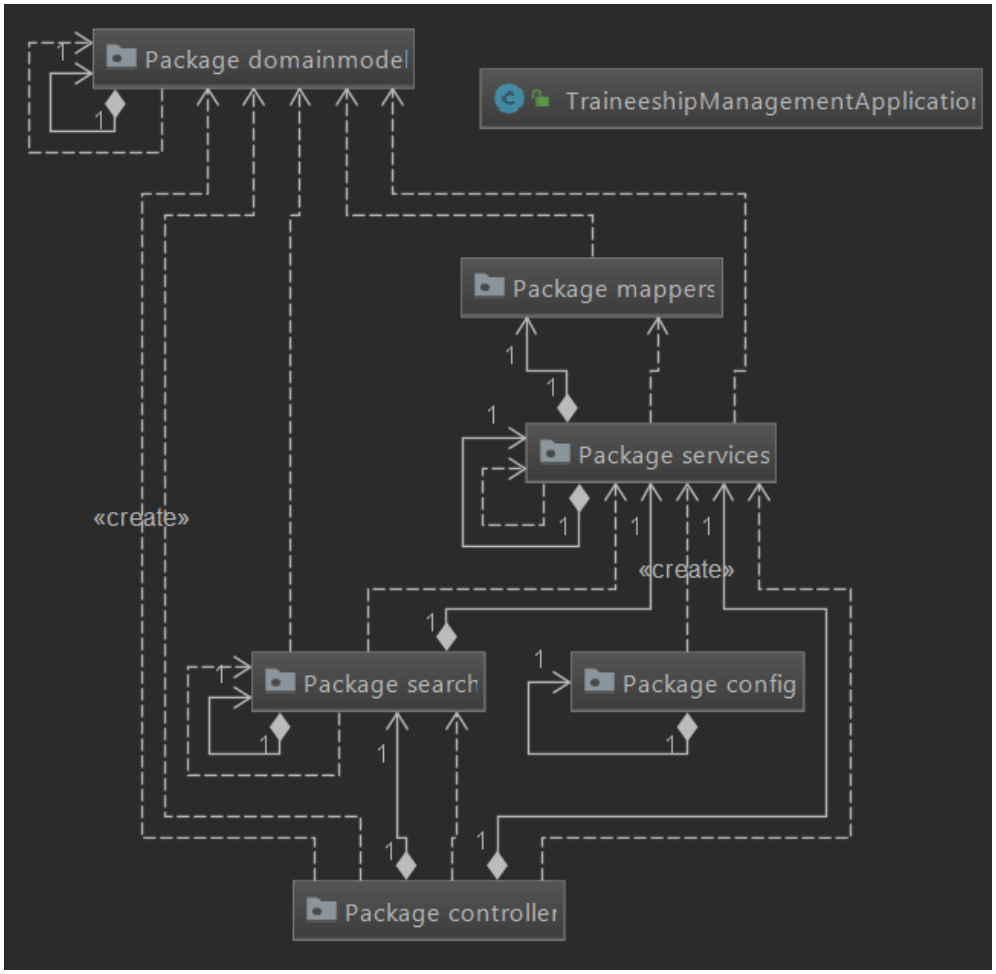
3.16 <showCompletedTraineeships>

Use case ID	16
Actors	The user, The committee
Pre conditions	The user has logged in and completed the whole process with a pass or fail mark depending on the evaluations.
Main flow of events	1. The use cases starts when the user wants to view the list with completed traineeships. 2. The user selects the "List Completed Traineeships" button. 3. The system displays the completed traineeships list with the fields Title of the traineeship, Company name, Student username, Dates and the committee decision on the traineeship.
Alternative flow 1	The user can go back to committee dashboard with the "Back to dashboard" button.
Post conditions	The user has viewed the list of completed traineeships.

4 Design

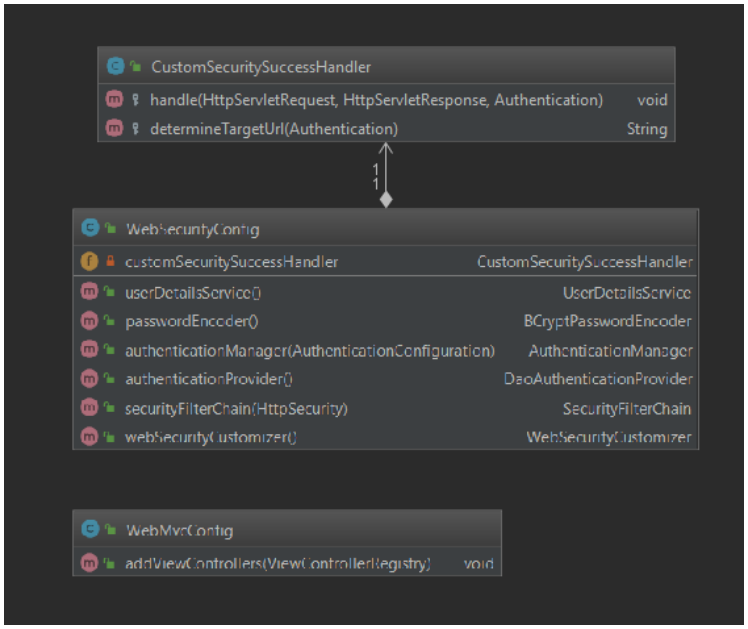
4.1 Architecture

UML of the packages:

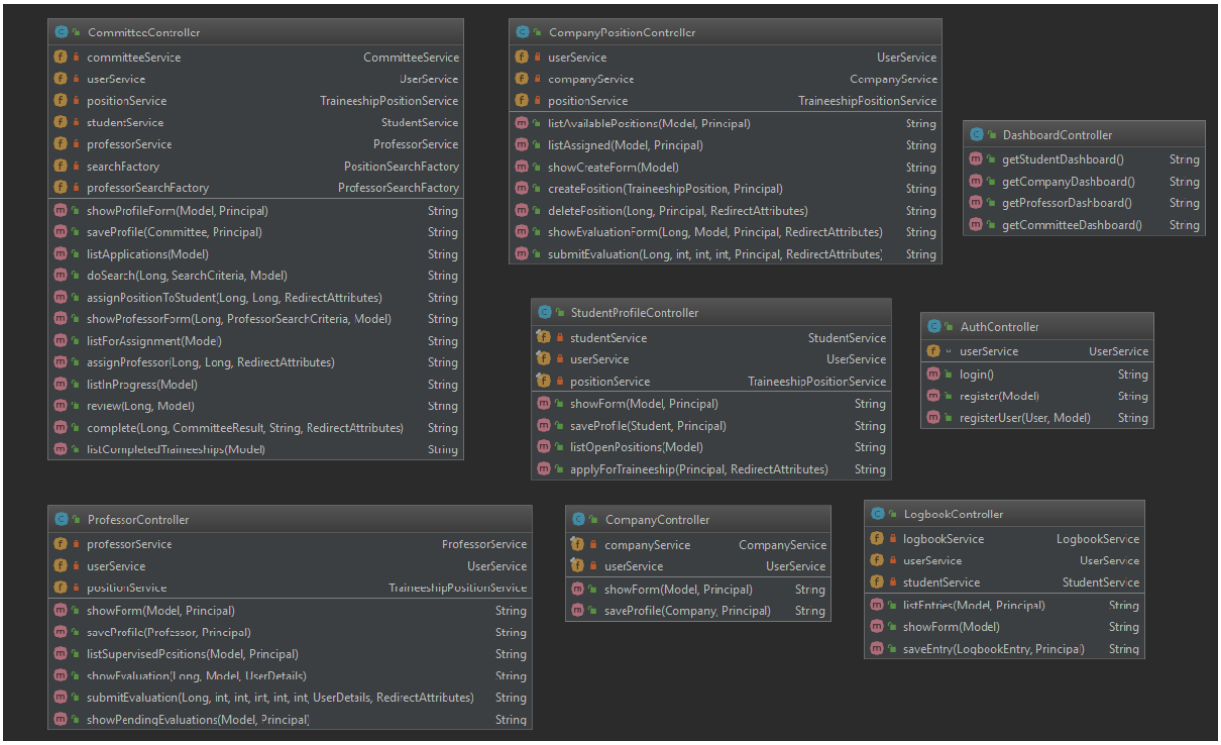


4.2 Design

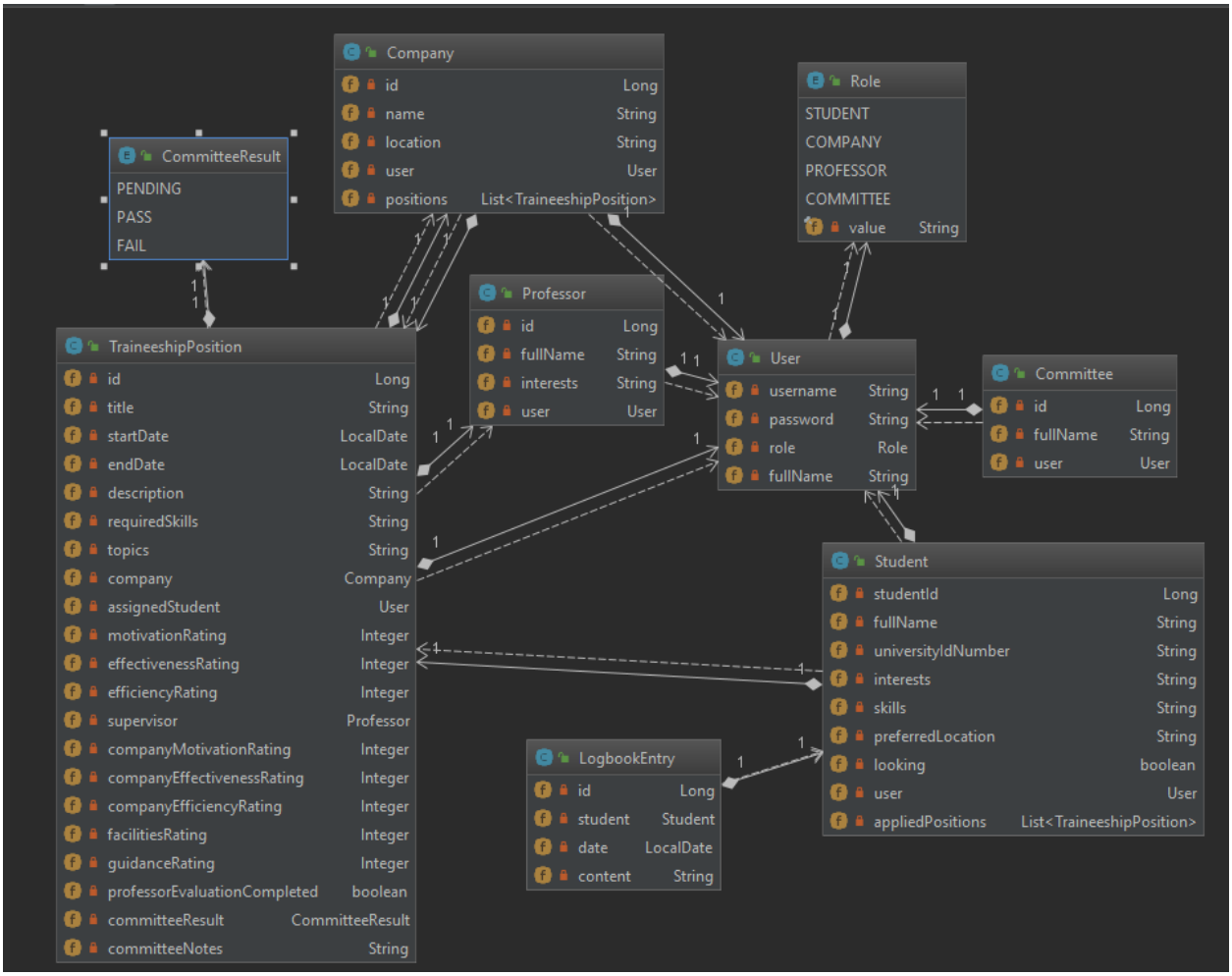
Config package:



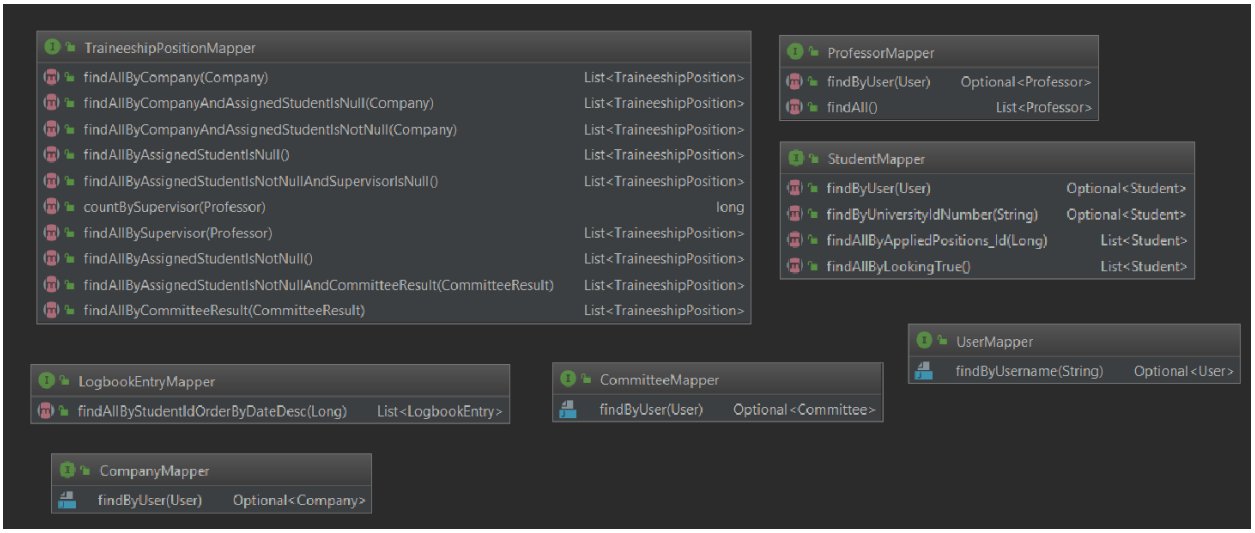
Controller package:



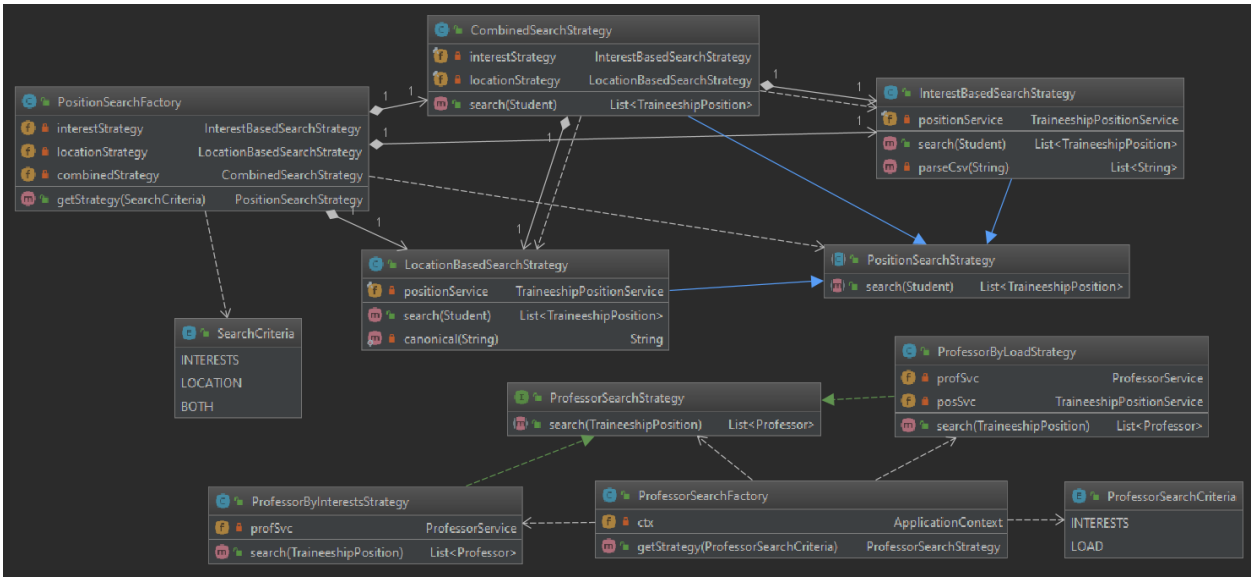
Domainmodel package:



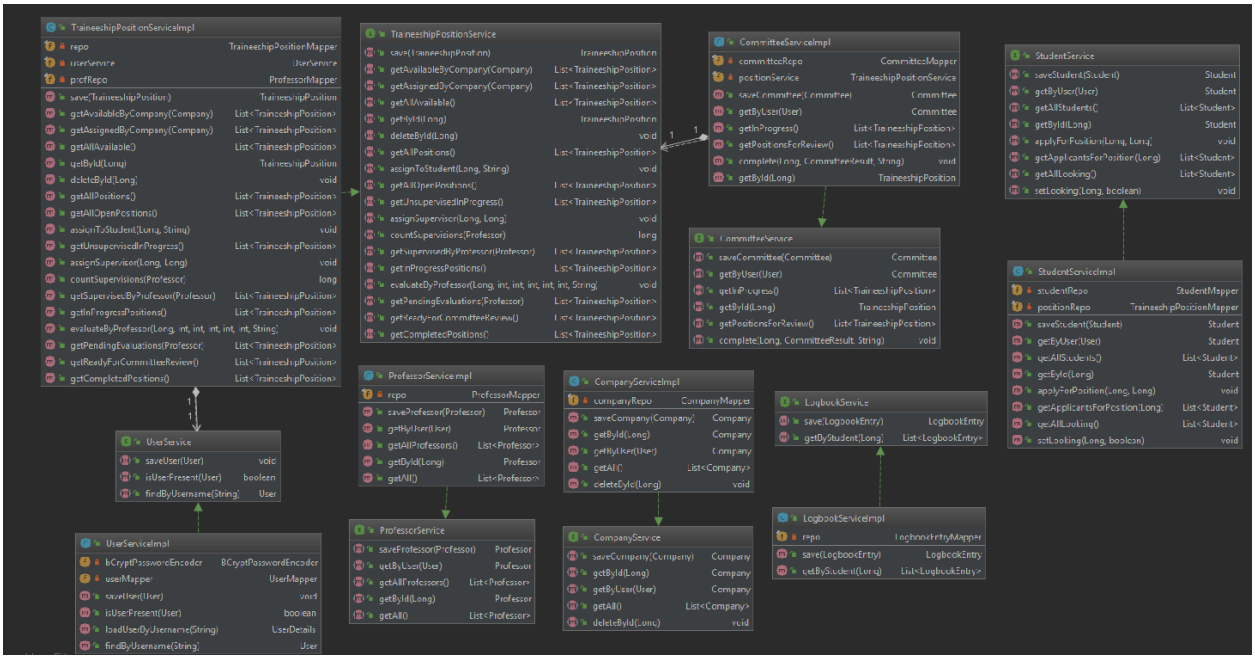
Mappers package:



Search package:



Services package:



Class Name: CustomSecuritySuccessHandler	
Responsibilities: ▪ After a user logs in, it picks the right page to show. It looks at the user's role (STUDENT, COMPANY, PROFESSOR, COMMITTEE) and builds the target URL. ▪ It sends the browser to that URL.	**Collaborations:** ▪ Extends Spring's SimpleUrlAuthenticationSuccessHandler and therefore plugs into the Spring Security login flow. ▪ Uses a Spring RedirectStrategy to do the redirect. ▪ Reads the user's roles from the Authentication object that Spring Security passes in.

Class Name: WebMvcConfig	
Responsibilities: ▪ Adds one simple view rule: when someone visits "/" (the root URL), show the homepage view. No extra controller code is needed.	**Collaborations:** ▪ Implements Spring's WebMvcConfigurer, so Spring calls it during start-up. ▪ Works with ViewControllerRegistry, which stores the path-to-view mapping.

Class Name: WebSecurityConfig	
Responsibilities: ▪ Sets up all security rules: • Which URLs anyone can reach (/, /login, /register, /save). • Which URLs need special roles, e.g., /company/** for COMPANY, /professor/** for PROFESSOR, and so on. ▪ Defines key security beans: • UserDetailsService that loads user data (UserServiceImpl). • BCryptPasswordEncoder for hashing passwords. • DaoAuthenticationProvider that	**Collaborations:** ▪ Uses the injected CustomSecuritySuccessHandler to redirect users after a successful login. ▪ Relies on UserServiceImpl for fetching user details from the database. ▪ Talks to many Spring Security classes (HttpSecurity, AuthenticationManager, DaoAuthenticationProvider, etc.) to build the full security setup.

joins the two. • SecurityFilterChain that ties the rules together. ▪ Customises login and logout pages and plugs in the custom success handler. ▪ Tells Spring to ignore static files under /images/**, /js/**, /webjars/**.	

Class Name: AuthController	
Responsibilities: ▪ Show the login page when a user goes to **/login.** ▪ Show the registration form when a user goes to **/register**, providing an empty User object and all possible Role values for the form. ▪ Handle form submission at **/save**: check if a user already exists, save a new user if not, and then show the login page again with a success or "already registered" message.	**Collaborations:** ▪ **UserService** to check for existing users and to save new users. ▪ **Spring MVC** components: uses Model to pass data to views and @ModelAttribute to bind form input to the User object.

Class Name: CommitteeController	
Responsibilities: ▪ Profile Management: • GET **/committee/profile**: load the logged-in committee member's profile into a form. • POST **/committee/profile**: save updates to that profile. ▪ Student Application Handling: • GET **/committee/applications**: list all students who applied. • POST **/committee/ applications/ {studentId} /search**: find suitable traineeship positions for a given	**Collaborations:** ▪ **CommitteeService, StudentService, TraineeshipPositionService** for loading and saving domain data. ▪ **Search components** (PositionSearchFactory and strategy classes) to find matching positions. ▪ **Spring MVC**: uses Model, Principal, @PathVariable, @RequestParam, and RedirectAttributes to handle input, security context, and redirection.

student, based on chosen criteria. • POST **/committee/applications /{studentId}/assign/ {positionId}**: assign a specific position to that student. ▪ Traineeship Review & Completion: • GET **/committee/ traineeships/{id}/review**: show details of a placed trainee for review. • POST **/committee/ traineeships/{id}/complete**: mark the traineeship as complete, recording the committee's decision. • GET **/committee/traineeships/ completed**: list all traineeships that have been completed.	

Class Name: CompanyController	
Responsibilities: ▪ GET **/company/profile**: display a form pre-filled with the company's existing data. ▪ POST **/company/profile**: save new or updated company information, then redirect to the company dashboard.	**Collaborations:** ▪ **CompanyService** to fetch and save the Company entity. ▪ **UserService** to look up the current user (by username from Principal). ▪ **Spring MVC**: uses Model to pass company data to the form and @ModelAttribute to bind form fields.

Class Name: CompanyPositionController	
Responsibilities: ▪ View & Manage Positions: • GET **/company/positions /assigned**: list all positions the company has assigned to students. • GET **/company/positions /create**: show a blank form to create a new	**Collaborations:** ▪ **UserService** to get the logged-in user. ▪ **CompanyService** to map that user to a Company. ▪ **TraineeshipPositionService** to create, find, delete, and list positions. ▪ **Spring MVC**: uses Model, Principal,

position. • POST **/company/positions /create**: save the new position under the current company. • GET **/company/positions /delete/{posId}**: delete a position if it belongs to the current company, showing a success or error flash message. ▪ Evaluate Student on a Position: • GET **/company/positions /{posId}/evaluate**: show a form where the company can rate and comment on the student's performance. • POST **/company/positions /{posId}/evaluate**: save those ratings and then go back to the assigned-positions list.	@PathVariable, @RequestParam, and RedirectAttributes.

Class Name: DashboardController	
Responsibilities: ▪ Serve the dashboard pages for each role: • GET **/student/dashboard** → student/dashboard view. • GET **/company/dashboard** → company/dashboard view. • GET **/professor/dashboard** → professor/dashboard view. • GET **/committee/dashboard** → committee/dashboard view.	**Collaborations:** ▪ **Spring MVC**: each method is a simple @GetMapping that returns the name of the view template.

Class Name: LogbookController	
Responsibilities: ▪ Show the list of logbook entries for the logged-in student.	**Collaborations:** ▪ **LogbookService** to fetch and save LogbookEntry objects.

▪ Display a form to add a new entry. ▪ Save a new entry with today's date and link it to the student.	▪ **UserService** to look up the current User from Principal. ▪ **StudentService** to find the Student tied to that User. ▪ Spring MVC components (@Controller, Model, @GetMapping, @PostMapping, @ModelAttribute, Principal) to handle web requests and views.

Class Name: ProfessorController	
Responsibilities: ▪ Show and save the professor's profile (full name, interests). ▪ List all traineeship positions this professor supervises. ▪ Display a form to rate a student's work on a supervised position. ▪ Submit and store those ratings.	**Collaborations:** ▪ **ProfessorService** to load and save Professor profiles. ▪ **UserService** to translate Principal or UserDetails into a User, then into a Professor. ▪ **TraineeshipPositionService** to fetch supervised positions and record evaluations. ▪ Spring MVC and Security (@Controller, @RequestMapping, Model, @AuthenticationPrincipal, ResponseStatusException, RedirectAttributes) for routing, form binding, and access control.

Class Name: StudentProfileController	
Responsibilities: ▪ Show and save the student's profile (name, skills, interests, preferred location). ▪ Display all open traineeship positions. ▪ Let the student apply for a traineeship (mark them as "looking").	**Collaborations:** ▪ **StudentService** to load, save, and update Student data (including setting "looking"). ▪ **UserService** to map Principal to User. ▪ **TraineeshipPositionService** to fetch open positions. ▪ Spring MVC (@Controller, Model, @GetMapping, @PostMapping, @ModelAttribute, RedirectAttributes)

	for web endpoints and form handling.

Class Name: Committee	
Responsibilities: ▪ Represent a committee member in the database with an **ID** and **full name**. ▪ Link that member to a Spring Security User account.	**Collaborations:** ▪ **JPA / Hibernate annotations** (@Entity, @Table, @Id, @GeneratedValue, @Column, @OneToOne, @JoinColumn) to map to the committees table. ▪ A **one-to-one** relationship with the User entity for login credentials and roles.

Class Name: CommitteeResult	
Responsibilities: ▪ Define the possible outcomes of a committee's decision on a traineeship: **PENDING**, **PASS**, or **FAIL**.	**Collaborations:** ▪ Used by **TraineeshipPosition** (and the committee workflow) to track status. ▪ Bound by Spring MVC in forms and controllers (e.g., as a @RequestParam in CommitteeController.complete). ▪ Interpreted by **CommitteeService** to finalize and store the decision.

Class Name: Company	
Responsibilities: ▪ Store a company's basic data: its ID, name, and location. ▪ Link that company to the user account that owns it. ▪ Keep a list of the traineeship positions the company offers.	**Collaborations:** ▪ One-to-one with **User** (the login account). ▪ One-to-many with **TraineeshipPosition** (the jobs it offers). ▪ Uses JPA annotations (@Entity, @OneToOne, @OneToMany, etc.) to map to the database.

Class Name: TraineeshipPosition	
Responsibilities: ▪ Hold all details of a traineeship: ID, title, start/end dates, description, required skills, topics. ▪ Track who is assigned (assignedStudent) and which professor supervises it. ▪ Store both professor and company evaluation ratings and notes. ▪ Compute an overall average rating for the position.	**Collaborations:** ▪ Many-to-one with **Company** (who offers the position). ▪ Many-to-one with **User** as the assigned student. ▪ Many-to-one with **Professor** as supervisor. ▪ Uses the **CommitteeResult** enum for final committee decision. ▪ Uses JPA mapping (@Entity, @ManyToOne, @Enumerated, etc.) to link to other entities.

Class Name: Role	
Responsibilities: ▪ Define the application's user roles: STUDENT, COMPANY, PROFESSOR, COMMITTEE.	**Collaborations:** ▪ Used by **User** to set its role field. ▪ Drives Spring Security authorities (via GrantedAuthority).

Class Name: Professor	
Responsibilities: ▪ Store a professor's profile data: ID, full name, and interests. ▪ Link the professor to their login account.	**Collaborations:** ▪ One-to-one with **User** (for authentication). ▪ Referenced by **TraineeshipPosition** as the supervising professor.

Class Name: LogbookEntry	
Responsibilities: ▪ Represent a single daily entry in a student's logbook, with date and content. ▪ Give each entry its own ID.	**Collaborations:** ▪ Many-to-one with **Student** (each entry belongs to one student). ▪ Uses JPA mapping (@Entity, @ManyToOne) to link to the student record.

Class Name: Student	
Responsibilities: ▪ Store a student's profile: ID, name, university number, interests, skills, preferred location, and a "looking" flag. ▪ Keep a list of which positions the student has applied to.	**Collaborations:** ▪ One-to-one with **User** (the student's login account). ▪ Many-to-many with **TraineeshipPosition** via the student_applications join table.

Class Name: User	
Responsibilities: ▪ Store login credentials and basic info: username, password, role, and full name. ▪ Implement Spring Security's UserDetails so it can be used for authentication. ▪ Provide the list of granted authorities based on its role.	**Collaborations:** ▪ Uses the **Role** enum to set its role. ▪ Connected (one-to-one) to each profile class: Company, Professor, Committee, and Student. ▪ Serves as assignedStudent in **TraineeshipPosition**. ▪ Integrates with Spring Security's GrantedAuthority mechanism.

Class Name: CommitteeMapper	
Responsibilities: ▪ Provides basic CRUD (create, read, update, delete) operations for Committee entities. ▪ Lets you look up a Committee by its associated User account, returning an Optional<Committee>.	**Collaborations:** ▪ Extends Spring Data's JpaRepository<Committee, Long> for built-in database methods. ▪ Works with the domain model classes Committee and User.

Class Name: CompanyMapper	
Responsibilities: ▪ Handles CRUD operations for Company entities. ▪ Finds a Company by its linked User, returning an Optional<Company>.	**Collaborations:** ▪ Extends JpaRepository<Company, Long> to inherit standard data-access methods. ▪ Uses the domain classes Company and User.

Class Name: ProfessorMapper	
Responsibilities: ▪ Offers CRUD operations for Professor entities. ▪ Supports finding a professor by their User account (findByUser) and listing all professors (findAll).	**Collaborations:** ▪ Extends JpaRepository<Professor, Long> for database access. ▪ Works with Professor and User domain classes.

Class Name: TraineeshipPositionMapper	
Responsibilities: ▪ Provides standard CRUD operations for TraineeshipPosition entities. ▪ Defines many finder methods to retrieve positions by company, student assignment status, supervisor, and committee result. ▪ Includes a custom JPQL query (findAllReadyForCommitteeReview) to get positions whose evaluations are complete and awaiting a committee decision.	**Collaborations:** ▪ Extends JpaRepository<TraineeshipPosition, Long> for basic data access. ▪ Works with domain classes TraineeshipPosition, Company, Professor, and the CommitteeResult enum.

Class Name: LogbookEntryMapper	
Responsibilities: ▪ Manages CRUD for LogbookEntry records. ▪ Retrieves all logbook entries for a given student ID, ordered by date descending (most recent first).	**Collaborations:** ▪ Extends JpaRepository<LogbookEntry, Long>. ▪ Uses the LogbookEntry domain model class.

Class Name: StudentMapper	
Responsibilities: ▪ Provides CRUD operations for Student entities. ▪ Finds a student by their User account	**Collaborations:** ▪ Extends JpaRepository<Student, Long>. ▪ Works with Student and User domain

or by university ID number. ▪ Lists students who have applied to a given position or who are currently looking for a placement.	classes, and leverages the student_applications join table for many-to-many links.

Class Name: UserMapper	
Responsibilities: ▪ Handles CRUD operations for User entities (keyed by username). ▪ Looks up a User by username, returning an Optional<User>.	**Collaborations:** ▪ Extends JpaRepository<User, String>. ▪ Works with the User domain model, which implements Spring Security's UserDetails.

Class Name: PositionSearchStrategy	
Responsibilities: ▪ Defines a single method search(Student student) that all concrete strategies must implement.	**Collaborations:** ▪ Serves as the base class for InterestBasedSearchStrategy, LocationBasedSearchStrategy, and CombinedSearchStrategy.

Class Name: InterestBasedSearchStrategy	
Responsibilities: ▪ Reads the student's interests and skills (from CSV strings). ▪ Loads all open positions and keeps only those where the student has **all** required skills **and** at least one interest matches the position's topics.	**Collaborations:** ▪ Calls TraineeshipPositionService.getAllAvailable() to fetch positions. ▪ Extends PositionSearchStrategy so it fits into the same strategy interface.

Class Name: LocationBasedSearchStrategy	
Responsibilities: ▪ Reads the student's preferred location and normalizes it (removing accents, spaces, case). ▪ If no location is given, returns an empty list. Otherwise, keeps only positions whose company location	**Collaborations:** ▪ Uses TraineeshipPositionService.getAllAvailable() for the initial list. ▪ Extends PositionSearchStrategy to plug into the same search API.

contains (or is contained in) the student's preference.	

Class Name: CombinedSearchStrategy	
Responsibilities: ▪ Runs both the interest-based and location-based searches. ▪ If the student has no preferred location, returns the interest-based results. ▪ Otherwise, returns only positions that appear in **both** interest and location lists.	**Collaborations:** ▪ Calls InterestBasedSearchStrategy.search() and LocationBasedSearchStrategy. search(). ▪ Extends PositionSearchStrategy so it works interchangeably with the other strategies.

Class Name: PositionSearchFactory	
Responsibilities: ▪ Chooses which PositionSearchStrategy to use based on a SearchCriteria value (INTERESTS, LOCATION, or BOTH).	**Collaborations:** ▪ Injects the three concrete strategies (interestStrategy, locationStrategy, combinedStrategy) via Spring. ▪ Returns them as the common PositionSearchStrategy interface.

Class Name: ProfessorByInterestsStrategy	
Responsibilities: ▪ Parses the position's topic CSV string into a list of topics. ▪ Filters all professors to those whose own interest CSV string shares at least one topic with the position.	**Collaborations:** ▪ Uses ProfessorService.getAll() to load every professor. ▪ Implements ProfessorSearchStrategy so it fits into the same interface as other search strategies.

Class Name: ProfessorByLoadStrategy	
Responsibilities: ▪ Sorts all professors by how many "in-progress" positions they currently supervise, in ascending order.	**Collaborations:** ▪ Uses ProfessorService.getAll() to get the list of professors. ▪ Uses TraineeshipPositionService.

	countSupervisions(professor) to measure each professor's current load. ▪ Implements ProfessorSearchStrategy so it can be swapped in via the factory.

Class Name: ProfessorSearchFactory	
Responsibilities: ▪ Chooses which ProfessorSearchStrategy bean to return based on a ProfessorSearchCriteria value.	**Collaborations:** ▪ Injects Spring's ApplicationContext and calls getBean() to fetch either ProfessorByInterestsStrategy or ProfessorByLoadStrategy. ▪ Works with ProfessorSearchCriteria to map each enum value to the correct strategy.

Class Name: ProfessorSearchCriteria	
Responsibilities: ▪ Defines the two ways the user can choose to find supervisors: • INTERESTS (match by topic overlap) • LOAD (match by current supervision load)	**Collaborations:** ▪ Used by controllers and ProfessorSearchFactory to decide which strategy to use.

Class Name: SearchCriteria	
Responsibilities: ▪ Defines how to find **positions** for students, with three options: • INTERESTS (match by skills/interests) • LOCATION (match by preferred location) • BOTH (combine both filters - intersection)	**Collaborations:** ▪ Used by PositionSearchFactory and controllers (in CommitteeController) to pick the right PositionSearchStrategy.

Class Name: CommitteeService	
Responsibilities: ▪ Declare methods to save a committee member, find a committee by its user account, list positions that are in progress or need review, and complete a committee decision (pass/fail with notes).	**Collaborations:** ▪ Works with the domain classes Committee, User, TraineeshipPosition, and CommitteeResult.

Class Name: CommitteeServiceImpl	
Responsibilities: ▪ Implements all CommitteeService methods: • Save or load Committee entities via the mapper. • Get "in progress" or "ready for review" positions by delegating to the position service. • Complete a decision by updating the TraineeshipPosition's committee result and notes.	**Collaborations:** ▪ Uses **CommitteeMapper** to read/write Committee objects. ▪ Calls **TraineeshipPositionService** to load, filter, and save position data.

Class Name: CompanyService	
Responsibilities: ▪ Declare methods to save a company profile, look up a company by ID or user, list all companies, and delete a company.	**Collaborations:** ▪ Works with the domain classes Company and User.

Class Name: CompanyServiceImpl	
Responsibilities: ▪ Implements CompanyService by: • Saving and deleting Company entities.	**Collaborations:** ▪ Uses **CompanyMapper** (a Spring Data JPA repository) for all database operations on Company.

• Finding companies by ID or by user account. • Returning a list of all companies.	

Class Name: LogbookService	
Responsibilities: ▪ Declare methods to save a logbook entry and fetch all entries for a given student (by student ID).	**Collaborations:** ▪ Works with the domain class LogbookEntry.

Class Name: LogbookServiceImpl	
Responsibilities: ▪ Implements LogbookService by: • Saving new LogbookEntry objects. • Retrieving a student's entries ordered from newest to oldest.	**Collaborations:** ▪ Uses **LogbookEntryMapper** to perform the actual save and query operations.

Class Name: ProfessorService	
Responsibilities: ▪ Declare methods to save a professor profile, look up a professor by user or by ID, and list all professors.	**Collaborations:** ▪ Works with the domain classes Professor and User.

Class Name: ProfessorServiceImpl	
Responsibilities: ▪ Implements ProfessorService by: • Saving Professor entities. • Fetching a professor by their linked user account or by their unique ID. • Returning all professors in the system.	**Collaborations:** ▪ Uses **ProfessorMapper** (Spring Data JPA repository) to load and store Professor data.

Class Name: StudentService

Responsibilities:	**Collaborations:**
▪ Declare methods for managing students: saving profiles, finding by user or ID, getting all students, handling applications, and tracking who is “looking” for a placement.	▪ Used by controllers or other services to work with Student data without knowing how it’s stored. ▪ Relies on a StudentMapper under the hood to access the database.

Class Name: StudentServiceImpl	
Responsibilities:	**Collaborations:**
▪ Implements all StudentService methods: • Save or update student records. • Look up a student by their user account or by ID. • Let a student apply for a position and save that link. • Fetch lists of applicants or students marked as “looking.”	▪ Uses StudentMapper to read and write Student entities. ▪ Uses TraineeshipPositionMapper to find positions when a student applies. ▪ Called by controllers to handle student-related web requests.

Class Name: TraineeshipPositionService	
Responsibilities:	**Collaborations:**
▪ Declare methods for managing traineeship positions: saving, deleting, listing by various filters (available, assigned, in progress, supervised, etc.), assigning students or supervisors, counting loads, and handling evaluations.	▪ Serves as the API for controllers and other services to work with TraineeshipPosition data. ▪ Depends on underlying mappers (TraineeshipPositionMapper) for data access.

Class Name: TraineeshipPositionServiceImpl	
Responsibilities:	**Collaborations:**
▪ Implements all position-service methods: • CRUD operations on positions.	▪ Uses TraineeshipPositionMapper to perform database queries and updates.

• Fetch lists based on company, student assignment, supervisor load, or committee status. • Assign a student or supervisor to a position. • Let professors submit ratings and track pending evaluations.	▪ Calls UserService to look up student accounts by username. ▪ Uses ProfessorMapper to find professors when assigning supervisors. ▪ Marked as @Service so Spring injects it where needed.

Class Name: UserService	
Responsibilities: ▪ Declare methods for user management: saving users, checking existence, and finding by username.	**Collaborations:** ▪ Used by authentication and registration controllers to handle user accounts. ▪ Relies on a UserMapper for persistence.

Class Name: UserServiceImpl	
Responsibilities: ▪ DImplements UserService and Spring Security's UserDetailsService: • Save new users, encoding passwords with BCrypt. • Check if a user already exists. • Load user details for login by username.	**Collaborations:** ▪ Uses UserMapper to read/write User entities. ▪ Uses BCryptPasswordEncoder to hash passwords. ▪ Registered as a Spring Security UserDetailsService so the framework calls it during authentication.